

Ma Sói Web App — Kế hoạch & Thiết kế

Last updated: 2026-05-21 · Scope: Biến `LUAT_MA_SOI.md` thành web app game Ma Sói chơi online nhiều người (phòng riêng theo mã + ghép trận người lạ có lắp bot), full bộ vai mở rộng, public có tài khoản, ưu tiên free-tier. Tech do assistant đề xuất.

Mục lục

- Kiến trúc tổng thể & Tech stack
- Nghịệp vụ & use-case (game flow thuần)
- Trải nghiệm ngoài trận & tính năng xã hội
- Tương tác & trải nghiệm trong trận
- Mô hình dữ liệu & API/Event contract
- Engine luật (server-authoritative)
- Backend: realtime, phòng, ghép trận, lắp bot
- Bot AI (người chơi máy)
- Tài khoản & xác thực
- Frontend web (giao diện người chơi)
- Visibility & chống gian lận
- Hạ tầng, triển khai & lộ trình MVP

1. Kiến trúc tổng thể & Tech stack

🟢 Đã chốt (2026-05-21)

Quyết định: TypeScript end-to-end trong **monorepo**, server game **authoritative**, realtime qua WebSocket. Ưu tiên hạ tầng **free-tier**.

- Ngôn ngữ:** TypeScript cho cả FE lẫn BE.
- Monorepo** (pnpm workspaces + Turborepo) gồm 4 gói: **engine** (luật thuần), **shared** (types + event contract), **server** (backend), **web** (frontend).
- Frontend:** React + TypeScript (Vite SPA).
- Backend realtime:** Node.js + Socket.IO (room, reconnect, phát sự kiện theo phòng).
- CSDL:** PostgreSQL (tài khoản, lịch sử ván, thống kê) + Redis (trạng thái phòng tạm, hàng đợi ghép trận, pub/sub khi scale).
- Hosting:** chọn nhà cung cấp nuôi WebSocket lâu dài có free-tier (Railway/Render/Fly.io) + Postgres & Redis free-tier; web tĩnh trên Vercel/Netlify.

Lý do: TS toàn phần cho phép **dùng chung engine luật + types** giữa FE/BE/bot — viết luật 1 lần, bot chạy đúng engine server. Socket.IO có sẵn phòng/kết nối lại, hợp game theo phòng. Server giữ trạng thái thật để chống gian lận (§11). Không chọn Next.js vì serverless khó nuôi WebSocket; không chọn Flutter Web vì không tái dùng được engine TS ở FE.

Cần verify sau: giới hạn kết nối WebSocket đồng thời + số giờ chạy của free-tier (ảnh hưởng số phòng song song); cân nhắc gộp Redis vào 1 instance để tiết kiệm khi còn nhỏ.

2. Nghiệp vụ & use-case (game flow thuần)

● Đã chốt (2026-05-21)

Quyết định: Flow ván theo đúng `LUAT_MA_SOI.md` (bản cuối), deterministic, không Quản trò người (luật + thứ tự lượt là trọng tài):

- Vòng đời ván: Lobby → ghép/tạo phòng → chia vai → Đêm 0 → Ngày 1 (không treo cổ) → Đêm 1 → Ngày 2 (bắt đầu treo cổ) → ... → kết thúc.
- Use-case lỗi: tạo/vào phòng theo mã, ghép trận người lạ, hành động đêm theo lượt vai, thảo luận, biểu quyết, khán giả khi chết.
- Bổ sung ở tầng ứng dụng (không đổi luật lõi):** pha thảo luận ban ngày diễn ra theo **lượt nói ngẫu nhiên** (xem §4) thay vì nói tự do đồng thời.

Lý do: luật đã được làm chặt qua nhiều vòng review (Q1–Q5), nên nghiệp vụ lỗi xem như khóa. Phần "tính tương tác/trải nghiệm" tách sang §3 (ngoài trận) và §4 (trong trận) để bàn riêng.

Cần verify sau: khi cài đặt engine, đối chiếu lại từng vai mở rộng với luật để không sót edge case.

3. Trải nghiệm ngoài trận & tính năng xã hội

● Đã chốt (2026-05-21)

Quyết định — v1 gồm:

- Sảnh (lobby):** nút "**Chơi nhanh**" (ghép trận người lạ) + "**Tạo phòng riêng**" (sinh mã/link mời để bạn bè vào thẳng) + danh sách phòng công khai.
- Hồ sơ + thống kê:** avatar, tên hiển thị, số ván, tỉ lệ thắng theo phe/vai.
- Bạn bè + trạng thái online:** thêm bạn, thấy bạn online, mời thẳng vào phòng.

Để phase sau: Xếp hạng/MMR + bảng xếp hạng; thành tích/huy hiệu + nhiệm vụ hằng ngày; lịch sử ván & xem lại (replay); cá nhân hóa cosmetic (avatar/khung/skin).

Lý do: v1 ưu tiên thứ giúp **vào trận nhanh** (sảnh + mời link) và **động lực chơi lại cơ bản** (hồ sơ/thống kê + bạn bè). MMR/replay/cosmetic tốn công và phụ thuộc lượng người chơi đủ lớn nên hoãn.

Cần verify sau: mời-quia-link đi kèm "Tạo phòng riêng" (mã phòng là đủ cho v1, link chỉ là bọc mã).

4. Tương tác & trải nghiệm trong trận

● Đã chốt (2026-05-21)

Quyết định — v1 gồm:

- Tương tác theo LƯỢT NÓI NGẪU NHIÊN (đặc trưng cốt lõi):** trong pha thảo luận ban ngày, hệ thống cho **mỗi người sống một lượt phát biểu** theo **thứ tự ngẫu nhiên** (đổi mỗi vòng), thay vì nói tự do đồng thời. Mỗi lượt có giới hạn thời gian.

- **Nhắn tin văn bản** là kênh tương tác chính (không voice ở v1).
- **Chat đa kênh**: kênh chung ban ngày (chỉ người sống), kênh **Sói** ban đêm, kênh **người chết** (khán giả).
- **Trạng thái trực quan**: đồng hồ pha đếm ngược, hiển thị ai đã hành động/đã bỏ phiếu (không lộ nội dung), đang tới lượt nói của ai.
- **Emote/biểu cảm nhanh** (👍🤔😬😡).
- **Chỉ điểm / buộc tội**: nhắm 1 người cho cả phòng thấy nghi ngờ trước khi vote.
- **Hiệu ứng + âm thanh** chuyển pha ngày/đêm và công bố cái chết.

Để phase sau: voice chat; lời cuối (last words); reaction gắn trên từng tin nhắn.

Lý do (lượt nói ngẫu nhiên + text): theo yêu cầu user — đi theo lượt giúp **để quan sát**, tránh hỗn loạn khi đồng người và khi có bot; **văn bản theo lượt** khiến mỗi phát ngôn gắn với một lượt rõ ràng → **dữ liệu dễ lưu/quản lý** (phục vụ chống gian lận §11, thống kê §3, và sau này replay). Đây là bổ sung ở tầng ứng dụng, **không đổi luật lõi**.

Cần verify sau: độ dài mỗi lượt nói + có cho "bỏ lượt/nói nhanh" không; cách xử lý khi tới lượt một **bot** (bot phát ngôn mẫu hay im lặng) — liên quan §8.

5. Mô hình dữ liệu & API/Event contract

🟢 Đã chốt (2026-05-21)

Quyết định:

- **Bền (PostgreSQL)**: `User`, `Friendship`, `ProfileStats`, `GameHistory` (kết quả ván + vai + phe thắng để thống kê).
- **Tạm thời (Redis)**: `RoomState`, `GameSession` (trạng thái ván đang chạy), `MatchQueue` (hàng đợi ghép trận). Hết ván thì đẩy tóm tắt sang `GameHistory`.
- **Event contract realtime** (namespaced, định nghĩa trong gói `shared`): client→server `room:create/join/leave`, `action:submit`, `vote:cast`, `chat:send`, `emote:send`, `accuse:set`, `speak:done`; server→client `room:update`, `game:phaseChanged`, `game:stateSync`, `turn:yours`, `chat:message`, `player:died`, `game:over`.
- **Nguyên tắc chốt**: server gửi cho mỗi client một **"góc nhìn đã lọc" (redacted view)** qua `game:stateSync` — chỉ chứa thông tin người đó được phép thấy (xem §11).

Lý do: tách bền/tạm giúp tận dụng Redis cho trạng thái nhịp nhanh (rẻ, hợp realtime) và Postgres cho dữ liệu cần lưu lâu. Contract nằm ở `shared` để FE/BE/bot dùng chung 1 nguồn type. "Redacted view" là cách an toàn nhất cho game suy luận.

6. Engine luật (server-authoritative)

🟢 Đã chốt (2026-05-21)

Quyết định: gói `engine` thuần TypeScript, **deterministic, không I/O**, là nguồn sự thật duy nhất của luật.

- API chính: `createGame(config, seed)`, `applyAction(state, action) → { state, events }`, `advancePhase(state) → { state, events }`, `checkWin(state) → winner | null`.
- **RNG có seed** (chia vai, thứ tự lượt nói ngẫu nhiên, lấp bot) để **test lặp lại được** và sau này **replay** được.
- Validate hành động theo "luật chọn mục tiêu hợp lệ" (Mục 6 của luật); resolver chết/dây chuyền + kiểm tra thắng theo Mục 9 (độc lập → Sói → Dân).
- Có **bộ test** phủ từng vai và các edge case (Phù thủy sau Sói, Bảo vệ chặn cản không chặn độc, Thợ săn bắn khi chết, cặp đôi chết theo, Sói Trắng...).

Lý do: tách engine khỏi mạng để **test kỹ** (đây là phần dễ sai nhất) và để **server lẫn bot dùng chung** — bot chỉ là một tác nhân gọi `applyAction`. Seed RNG mở đường cho replay (§3 phase sau) mà không phải thiết kế lại.

7. Backend: realtime, phòng, ghép trận, lắp bot

● Đã chốt (2026-05-21)

Quyết định:

- **Server WebSocket (Socket.IO)** giữ trạng thái thật, chạy **đồng hồ pha** phía server.
- **Phòng riêng (mã):** chủ phòng tạo, chia sẻ mã/link; không tự lắp bot (chủ phòng có thể thêm bot thủ công nếu thích).
- **Ghép trận người lạ:** hàng đợi; cần **tối thiểu 1 người thật** để mở; **đếm ngược ~30–45s**; hết giờ mà chưa đủ người (theo cấu hình ván) thì **lắp bot** cho đủ. Trong lúc chờ, người thật vào thêm được ưu tiên; bot chỉ lắp ở thời điểm chốt.
- **Mất kết nối:** giữ slot + cho **re-join** trong thời gian ngắn; quá hạn thì xử như rời ván (theo Mục 11.3 luật: coi như chết, kích hoạt hiệu ứng).

Lý do: chờ ngắn để vào trận nhanh (trải nghiệm mượt), bot lắp để không "treo" hàng đợi khi vắng người. Đồng hồ + trạng thái ở server là bắt buộc cho tính authoritative (§11).

Cần verify sau: số người mục tiêu mỗi ván ở mode ghép nhanh (đề xuất mặc định 8); giới hạn WebSocket đồng thời của free-tier (§1).

8. Bot AI (người chơi máy)

● Đã chốt (2026-05-21)

Quyết định — mục tiêu: bot suy luận theo thông tin (không chỉ random hợp lệ):

- Ban đêm: hành động theo vai một cách hợp lý (Sói nhắm vai nghi là Dân chức năng; Tiên tri soi người khả nghi; Bảo vệ giữ người giá trị; Phù thủy cân nhắc cứu/độc...).
- Ban ngày, khi **tối lượt nói**: phát ngôn dựa trên nghi ngờ (ai đang bị nghi, ai vote ai, ai chết đêm qua) bằng câu sinh theo ngữ cảnh; biểu quyết theo suy luận.
- Bot nhận **đúng "redacted view"** như người thật (không gian lận thông tin) và đi qua chính `engine` luật.
- **Triển khai theo bậc (khớp §12):** bản đầu = bot **heuristic vững** (hợp lệ + nghi ngờ cơ bản + phát ngôn mẫu theo ngữ cảnh); **nâng dần chiều sâu suy luận** qua các đợt.

Lý do: user muốn trận có bot vẫn hấp dẫn, nên đặt mục tiêu bot biết suy luận. Nhưng để có bản chạy sớm, bắt đầu bằng heuristic rồi cải thiện — tránh sa lầy vào AI phức tạp trước khi game chạy được.

Cần verify sau: mức độ phát ngôn của bot đến đâu là "đủ giống người" mà không gây nhiễu; có cần gắn nhãn bot hay ẩn danh hoàn toàn.

9. Tài khoản & xác thực

● Đã chốt (2026-05-21)

Quyết định — v1 hỗ trợ cả 3 cách:

- **Khách (guest):** vào chơi ngay bằng tên hiển thị, hồ sơ tạm (thống kê không lưu lâu dài).
- **Google OAuth:** đăng nhập nhanh.
- **Email + mật khẩu:** cho ai không dùng Google (mật khẩu **hash** bằng argon2/bcrypt).
- **Nâng cấp guest → tài khoản thật** để giữ lại thống kê.
- Dùng thư viện auth gọn (Lucia/Auth.js), phiên an toàn (cookie HttpOnly), lưu user ở Postgres.

Lý do: user chọn cả ba — guest giảm rào cản vào chơi (hợp mode chơi với bạn), Google nhanh, email phổ quát. Google OAuth miễn phí nên không đội chi phí free-tier.

Cần verify sau: email verification có thể **hoãn** (chỉ cần khi đăng ký email+mật khẩu); chống lạm dụng guest (rate-limit tạo phòng).

10. Frontend web (giao diện người chơi)

● Đã chốt (2026-05-21)

Quyết định: React + TypeScript (Vite), **mobile-first**, tiếng Việt.

- **State realtime:** đồng bộ qua Socket.IO; chỉ render từ "redacted view" server gửi (không tự suy luận trạng thái ẩn).
- **Quản lý state nhẹ:** Zustand (hoặc Context) cho client state; **TailwindCSS** để dựng UI nhanh, gọn.
- **Màn hình:** Lobby (Chơi nhanh / Tạo phòng / danh sách phòng), Phòng chờ (mã mời, danh sách người + bot), Màn chơi (pha + đồng hồ, lượt nói hiện tại, chọn mục tiêu kín, chat đa kênh, emote, chỉ điểm, biểu quyết + "Tha", khán giả khi chết), Hồ sơ/Bạn bè.
- Hiện thực toàn bộ tính năng đã chốt ở §3 và §4.

Lý do: Vite SPA nhẹ, khởi động nhanh; mobile-first vì người Việt chơi Ma Sói chủ yếu trên điện thoại; Tailwind tăng tốc dựng UI cho 1 người làm. Render thuần từ server-view để không rò rỉ thông tin (§11).

11. Visibility & chống gian lận

● Đã chốt (2026-05-21)

Quyết định: server-authoritative tuyệt đối — client không bao giờ biết thông tin nó không được phép thấy.

- Mỗi `game:stateSync` là **góc nhìn riêng đã lọc** của từng người: vai của người khác **không** gửi xuống; kết quả soi/nạn nhân chỉ gửi cho đúng vai; tin nhắn kênh Sói/người chết chỉ phát cho đúng nhóm.
- Mọi hành động/biểu quyết **validate phía server** theo engine; client gửi ý định, server mới là nơi quyết.
- Lượt nói (§4): server kiểm soát ai đang được phép gửi tin ở pha nào.

Lý do: game suy luận sống nhờ thông tin ẩn — nếu lộ qua payload thì hỏng game. Đặt toàn bộ sự thật ở server cũng chặn chỉnh sửa client. Đây là lý do then chốt chọn kiến trúc authoritative ở §1.

Cần verify sau: rà từng event đảm bảo không "vô tình" kèm field nhạy cảm (vai, mục tiêu) khi serialize.

12. Hạ tầng, triển khai & lộ trình MVP

● Đã chốt (2026-05-21)

Quyết định — lộ trình theo bậc (core trước, mở rộng dần):

- **GĐ1 — MVP chơi được với bạn:** monorepo (engine/shared/server/web); **engine 6 vai core + test**; server realtime + **phòng riêng theo mã**; FE màn chơi cơ bản (pha, lượt nói ngẫu nhiên, chat đa kênh, biểu quyết + "Tha", emote, chỉ điểm); **auth guest**. → Chơi được với bạn qua mã.
- **GĐ2 — ghép trận & tài khoản:** hàng đợi ghép trận + **lắp bot (heuristic)**; **Google + email auth**, nâng cấp guest; **hồ sơ + thống kê; bạn bè + online**.
- **GĐ3 — mở rộng nội dung:** bổ sung dần **vai mở rộng** (Cupid, Sói Trắng, Sói Tiên tri, Sói Nguyên, Thầy đồng, Trưởng làng, Già làng, Thằng Ngổ, Kẻ Thối Sáo; Cô bé chỉ chế độ trực tiếp); **nâng bot suy luận sâu hơn**; hiệu ứng + âm thanh đầy đủ.
- **Sau:** MMR/xếp hạng, thành tích/nhiệm vụ, replay, cosmetic, voice chat.

Hạ tầng (free-tier): server trên Railway/Render/Fly (free), Postgres + Redis free-tier (cân nhắc gộp lúc còn nhỏ), web trên Vercel/Netlify; **CI** GitHub Actions (lint + test engine); logging cơ bản.

Lý do: có **bản chơi được sớm** (GĐ1) để kiểm chứng vòng gameplay và trải nghiệm lượt-nói trước khi đổ công vào ghép trận/bot/social. Cắt theo bậc giảm rủi ro và hợp người làm ít.

Cần verify sau: chọn cụ thể nhà cung cấp free-tier sau khi đo nhu cầu WebSocket; ngưỡng nâng cấp lên gói trả phí.

Lịch sử cập nhật

Ngày	Section	Thay đổi
2026-05-21	(init)	Tạo skeleton + đề xuất 10 section. Chốt scope: 2 mode (phòng mã + ghép trận lắp bot), full vai, public có tài khoản, tech do assistant đề xuất
2026-05-21	\$1	Đề xuất tech stack (TS monorepo, React+Vite, Node+Socket.IO, Postgres+Redis)
2026-05-21	\$1	Chốt ● tech stack + ràng buộc ưu tiên free-tier
2026-05-21	(cấu trúc)	Thêm \$3 (trải nghiệm ngoài trận & xã hội) và \$4 (tương tác trong trận) theo phản hồi user: 10 section cũ thiên về luật, thiếu nghiệp vụ game + tính tương tác. Renummer còn 12 section
2026-05-21	\$2	Chốt ● nghiệp vụ theo luật bản cuối; bổ sung thảo luận ban ngày theo lượt nói (tàng ứng dụng)
2026-05-21	\$3	Chốt ● v1: Sảnh (chơi nhanh + tạo phòng/mã mời), Hồ sơ+thống kê, Bạn bè+online. Hoãn: MMR, thành tích, replay, cosmetic
2026-05-21	\$4	Chốt ● v1: lượt nói ngẫu nhiên + text, chat đa kênh, trạng thái/đồng hồ, emote, chỉ điểm, hiệu ứng+âm thanh. Hoãn: voice, last-words, reaction theo tin nhắn. Lý do lượt-nói: dễ quan sát + data để quản lý
2026-05-21	\$5,\$6,\$10,\$11	Chốt ● các section kỹ thuật (user giao assistant đề xuất): data model Postgres+Redis & event contract redacted-view; engine luật thuần deterministic + seed RNG + test; frontend React/Vite mobile-first + Tailwind; visibility server-authoritative
2026-05-21	\$7	Chốt ● ghép trận chờ ~30–45s rồi lắp bot (tối thiểu 1 người thật); phòng riêng không auto-bot; mất kết nối cho re-join, quá hạn xử như rời
2026-05-21	\$8	Chốt ● bot mục tiêu suy luận theo thông tin, triển khai theo bậc (heuristic trước, sâu dần); bot dùng redacted-view + engine như người

Ngày	Section	Thay đổi
2026-05-21	§9	Chốt ● v1 hỗ trợ guest + Google + email; nâng cấp guest→tài khoản; argon2/bcrypt; email verify hoãn
2026-05-21	§12	Chốt ● lộ trình 3 giai đoạn (MVP phòng-mã 6 vai → ghép trận+bot+tài khoản → mở rộng vai+bot sâu); free-tier deploy + CI
2026-05-21	(toàn bộ)	12/12 section đã ●. PLAN sẵn sàng chuyển Phase 2 (PROGRESS.md)